

Paper Summary

Probabilistic Plan Recognition Using Off-the-Shelf Classical Planners

Miquel Ramírez & Hector Geffner — AAAI-10 (2010)

What Problem Does It Solve?

Plan recognition is the problem of observing an agent's actions and inferring which goal it is pursuing. It is the reverse of planning: in planning you are given a goal and must find actions; in plan recognition you are given actions and must find the goal.

Prior approaches required a hand-built **plan library** — a pre-compiled catalogue of every possible plan. This paper removes that requirement entirely by using a standard AI **planning engine** as a black box.

Core Idea

For each candidate goal G , run a classical planner twice:

Planning Problem	What It Computes
$G + 0$ (compliant)	Cheapest plan for G that includes the observed actions
$G + \bar{0}$ (non-compliant)	Cheapest plan for G that excludes the observed actions

The **cost difference** $\Delta(G, O) = c(G, O) - c(G, \bar{O})$ measures how “surprising” the observations are if the agent is really pursuing G . A small (or negative) difference means the agent's behaviour is consistent with G ; a large difference means it is not.

The Probabilistic Model

Using Bayes' Rule:

$$P(G \mid O) = \alpha \cdot P(O \mid G) \cdot P(G)$$

Where the **likelihood** is a Boltzmann (softmax) distribution over costs:

$$P(O \mid G) \propto \exp\{-\beta \cdot c(G, O)\}$$

$$P(O \mid \bar{O}) \propto \exp\{-\beta \cdot c(G, \bar{O})\}$$

The ratio of these two quantities depends only on $\Delta(G, O)$, so the planner does not need to be modified — it just needs to return optimal costs for the two transformed problems.

The β Parameter

β controls how sharply the model penalises deviations from the optimal path. The paper does not specify its value for experiments; this is a key ambiguity practitioners must resolve.

PDDL Domain Transformation

To make a standard planner produce plans that *comply with* or *avoid* the observations, the authors define a simple domain transformation (Definition 2 / Proposition 3):

1. For each observed action $a \in O$, add a new fluent p_a .
2. Modify the action effects so that p_a is set to true when action a is executed in its correct position in the observation sequence.
3. The **compliant goal** $G + O$ appends p_a (last observed action) to the original goal.
4. The **non-compliant goal** $G + \bar{O}$ appends $\neg p_a$ to the original goal.

This ensures any standard STRIPS planner (e.g. Fast Downward, HSP*) can be used without modification.

Experimental Results

The method was tested on **six domains** using three planners: - **HSP*_F** — optimal planner (gold standard) - **Anytime LAMA** — satisficing planner (up to 240 s) - **Greedy LAMA** — satisficing planner (up to 120 s)

Domain	Type	Size ($ G $)	Notes
Block Words	Benchmark	20	Blocks World, spell a word
IPC-Grid	Benchmark	7.5 avg	Grid transport with keys
Logistics	Benchmark	10	Packages via trucks/planes
Intrusion Detection	Real-world	15	Hacker attack modelling
Campus	Real-world	2	Student activity tracking
Kitchen	Real-world	3	Meal preparation

Key metrics: - **Q** — fraction of problems where the hidden goal ranked as most likely (accuracy) - **S** — average number of goals tied as most likely (specificity; lower is better)

Key findings: - Anytime LAMA matched the quality of the optimal planner in a fraction of the time. - Greedy LAMA was sometimes an order of magnitude faster with only a small quality drop. - Q reached 1.0 (perfect accuracy) at 50–100% observation coverage in most domains. - The approach handles “noisy” agents who do not take optimal paths, unlike the earlier (2009) formulation.

Two Key Advantages Over Prior Work

1. **No plan library needed** — only a domain theory (PDDL file) is required. This is more flexible and general.
 2. **Off-the-shelf planners** — the 2009 predecessor required modifying the planner; this paper does not.
-

Limitations

- Restricted to **deterministic actions** and **full initial-state information** (classical planning setting).
 - The **Boltzmann approximation** assumes the most likely plan dominates the sum over all plans — reasonable but not always exact.
 - β is unspecified, making exact replication difficult.
-

Why It Matters

The paper bridges **planning** and **probabilistic inference**, showing that a core AI inference task (goal recognition) can be reduced to two calls to a standard planning engine. This makes the approach both theoretically elegant and practically scalable — handling domains with hundreds of actions and fluents efficiently.